

Information Retrieval

Programming Assignment I

Group Name: Information Retrieval Group Project

Choudhary Asmi Aashay – 23CS30061

G. Sri Mahitha Pranavi – 23CS30021

Sanjana Kothi – 23CS10063

1 Indexing

The indexing stage takes the preprocessed collection as input. Each document in this file begins with an `.I` line containing its document identifier, followed by an `.S` line and the processed tokens. The aim is to build an inverted index that maps every distinct token to the documents in which it occurs. The input file is named `Information_Retrieval_Group_Project_processed.all`.

1.1 Reading the processed collection

The file is read sequentially rather than loading the complete collection into memory. While reading a document, its tokens are stored in a set. This removes repeated occurrences of the same token within that document, since a Boolean index only needs to record whether a term is present. When the next `.I` tag is encountered, the current document is added to the postings list of every term in its set. The final document is added after the end of the file is reached. The parser also checks the structure of the input. It reports an error for a malformed document identifier, a repeated identifier, a missing or repeated `.S` tag, an unknown tag, or tokens occurring outside an `.S` section. These checks help prevent a damaged processed file from producing an incorrect index.

1.2 Building the postings lists

The index is represented by a dictionary whose key is a term and whose value is its postings list. Document identifiers are appended as the documents are read. The processed collection is arranged in increasing document id order, so the postings lists are produced in ascending order. A set of document identifiers is maintained separately to detect duplicates and to obtain the maximum document identifier. Before writing the file, the vocabulary is sorted lexicographically. The first line of the index contains the vocabulary

size and the maximum indexed document identifier. Every remaining line contains one term followed by its comma separated postings list. The format is therefore:

```
vocabulary_size, maximum_docid  
term docid1,docid2,docid3
```

For example, a possible entry is:

```
experimental 1,7,9,21,27
```

The generated file is named `Information_Retrieval_Group_Project_cran.index`.

1.3 Correctness checks

The completed index is read again and checked against the in-memory postings. The check verifies that the header is correct, terms are in strict lexicographic order, postings contain unique document identifiers in ascending order, every identifier is within the valid range, and the number of term lines matches the vocabulary size. The index is accepted only if all these checks pass.

1.4 Complexity

Let T be the number of processed token occurrences, V the vocabulary size, and P the total number of term–document pairs. Reading the collection and constructing the postings lists takes $O(T)$ expected time. Sorting the vocabulary takes $O(V \log V)$ time, and writing the postings takes $O(P)$ time. The overall expected time complexity is therefore $O(T + V \log V + P)$, while the index itself requires $O(P)$ space.

2 Retrieval

The retrieval stage uses the inverted index built in Section 1 to answer Boolean queries. A query is an expression formed from terms and the operators AND, OR and NOT. Retrieval is an exact match process: a document is returned if and only if it satisfies the Boolean expression, and no ranking is produced.

Answering such a query reduces to performing set operations on postings lists. The postings list of a term is the list of document identifiers in which the term occurs, and by construction every postings list is stored in ascending order of document identifier. This sorted order is the property that all the algorithms below rely on, because it allows a conjunction or a disjunction to be evaluated by a single coordinated scan instead of by repeated lookups.

2.1 The two-pointer merge

The baseline method for combining two postings lists is a two-pointer scan. One pointer is placed at the start of each list and the two lists are consumed together.

For the intersection (**AND**), the document identifiers under the two pointers are compared. If they are equal, the identifier belongs to the result and both pointers are advanced. If they are unequal, only the pointer standing on the smaller identifier is advanced, because that identifier cannot possibly appear later in the other list. The scan stops as soon as either list is exhausted, since no further matches are possible.

For the union (**OR**), the smaller of the two identifiers is appended to the result and its pointer is advanced. When the two identifiers are equal, the identifier is appended once and both pointers are advanced, which keeps the result free of duplicates. When one list is exhausted, the remaining tail of the other list is copied to the result unchanged.

Each comparison advances at least one pointer, so both operations examine every element of both lists at most once. The running time is $O(|A| + |B|)$, where $|A|$ and $|B|$ are the sizes of the two postings lists.

2.2 Optimised binary search merge

The two-pointer scan treats both lists symmetrically, and therefore pays for the length of the longer list even when the shorter one is tiny. When the lists are highly asymmetric, that is when $|A| \ll |B|$, it is cheaper to iterate over the short list A and to test membership of each of its identifiers in the long list B by binary search.

Range narrowing. A plain binary search would search the whole of B for every element of A , which repeats work unnecessarily. Since both lists are sorted, the elements of A are probed in increasing order, so each successive identifier can only be located further to the right in B than the previous one. The search therefore records the position `pos` at which the previous probe landed and sets the lower bound of the next search to `pos+1`. The searchable window shrinks monotonically as the scan proceeds, and the beginning of B is never rescanned.

With $|A|$ probes, each costing at most $O(\log |B|)$ comparisons, the intersection runs in $O(|A| \log |B|)$ time. This is an improvement over $O(|A| + |B|)$ precisely when $|A| \log |B| < |A| + |B|$, that is when A is much shorter than B .

Use for the union. The same idea can be applied to **OR**, but less profitably. The result is initialised with a copy of the larger list B , and every element of A is binary searched in it; an element that is not found is inserted at the position returned by the search. The search itself is logarithmic, but insertion into a Python list is $O(N)$ in the worst case because the following elements must be shifted. The method is therefore faster than the linear merge only when A is extremely small, so that very few insertions occur.

2.3 Skip pointer merge

The second alternative augments a postings list with skip pointers. At regular intervals the list stores a forward pointer to a later position, so that a block of entries can be

bypassed in one step. The interval is chosen as $\sqrt{L}$ for a list of length L , which balances the two competing costs: a longer interval gives fewer skips to follow but smaller savings per skip, while a shorter interval gives more pointers to store and check.

During an intersection, whenever the current identifier of A is larger than the current identifier of B , the pointer in B must move forward. Before advancing one position at a time, the algorithm inspects the skip target of B . If the identifier at that target is still less than or equal to the current identifier of A , the target cannot overshoot any possible match, and the pointer jumps straight to it, discarding every intermediate entry without examining it. If the target would overshoot, the pointer advances by one as in the ordinary merge. The symmetric test is applied when the pointer in B is ahead of the pointer in A .

In the worst case no skip is ever taken and the cost remains $O(|A| + |B|)$, but on typical queries a large fraction of the longer list is bypassed and the observed cost is substantially lower.

Why skips do not help the union. A union must output every document that occurs in either list. There is consequently no element that can be discarded without inspection, and skipping an element would mean omitting it from the result. Skip pointers are therefore not applicable to **OR**, and the implementation falls back on the two-pointer merge for that operator.

2.4 Summary of complexities

Table 1 collects the asymptotic costs of the three methods.

Method	AND	OR
Two-pointer merge	$O(A + B)$	$O(A + B)$
Binary search merge	$O(A \log B)$	$O(A \log B + A B)$ worst case
Skip pointer merge	$O(A + B)$ worst case	Not applicable

Table 1: Asymptotic cost of combining two postings lists of sizes $|A|$ and $|B|$, with $|A| \leq |B|$. The union entry for the binary search method includes the cost of shifting elements on insertion.

2.5 Experimental evaluation

The three methods were compared on a deliberately asymmetric pair of terms drawn from the index, so that the effect of the size imbalance would be visible. The short term is **abbrevi**, whose postings list holds a single document, and the long term is **flow**, whose postings list holds 730 documents. In the notebook these two lists are held in the variables `small_docs` and `big_docs_list` respectively. Each measurement is the total time for 10,000 repetitions of the operation, which keeps the timing above the resolution of the clock and averages out interference from the operating system. The results are

given in Table 2, with the speedup reported relative to the two-pointer merge for the same operator.

Operation	Method	Time (s)	Speedup
AND	Two-pointer merge (<code>t_two_pointer</code>)	0.07450	—
AND	Optimised binary search (<code>t_binary</code>)	0.00282	$\approx 26.4\times$
AND	Skip pointer merge (<code>t_skips</code>)	0.06054	$\approx 1.2\times$
OR	Two-pointer merge (<code>t_or_two_pointer</code>)	0.42751	—
OR	Binary insertion (<code>t_or_binary</code>)	0.00758	$\approx 56.4\times$
OR	Skip pointer union	Not applicable	—

Table 2: Time for 10,000 runs on the postings lists of **abbrevi** (1 document) and **flow** (730 documents).

2.6 Discussion of the results

The measurements agree with the analysis. For the intersection, the binary search method is about 26 times faster than the linear merge. With $|A| = 1$ and $|B| = 730$ it performs a single search of roughly $\log_2 730 \approx 10$ comparisons, whereas the two-pointer scan must walk through the long list until it passes the single matching identifier.

The skip pointer method gives only a modest gain of about $1.2\times$ on this pair. This is expected rather than disappointing: with an interval of $\sqrt{730} \approx 27$, the scan can bypass at most twenty-seven entries per jump and still has to follow the chain of skip pointers and test each target. Skips reduce the constant factor of a linear scan; they do not change its order, so they cannot compete with a logarithmic search at this level of imbalance. Their value appears when the two lists are of comparable and moderate size, where the binary search overhead is no longer repaid.

The union shows the largest gap, roughly $56\times$. The two-pointer merge builds its result by appending all 730 identifiers of the long list one at a time, while the binary insertion method copies that list once and performs a single search and at most one insertion for the lone element of the short list. The cost of the bulk copy is much lower than the cost of the element-by-element rebuild.

2.7 Conclusion

Three conclusions follow from the experiments.

1. For AND queries on highly asymmetric postings lists, that is when $|A| \ll |B|$, binary search with range narrowing is the fastest of the three methods.
2. For AND queries on lists of comparable and moderate size, skip pointers give the best balance, since they reduce the number of entries examined without incurring the overhead of a logarithmic search per element.

3. For **OR** queries the linear two-pointer merge is the general choice, because every element of both lists must appear in the output. The exception is the case of one very short list, where binary insertion is faster in practice as it avoids rebuilding the long list element by element.

A practical implementation can therefore choose its merge strategy at query time by comparing the lengths of the two postings lists.